0072. Edit Distance

Medium · 2-D Dynamic Programming
leetcode.com/problems/edit-distance

Problem

- Minimum insertions, deletions and replacements to turn word1 into word2.

Intuition

- Dp[i][j] = cost to convert a[0..i) into b[0..j).
- If the last chars match: dp[i-1][j-1]. Else 1 + min(replace dp[i-1][j-1], delete dp[i-1][j], insert dp[i][j-1]).

Brute Force

- Recursion over three choices: O(3^(m+n)).

Optimal Approach

- Dp[i][0] = i, dp[0][j] = j.
- Fill with the rule above.

Worked Example

```
"horse" -> "ros".

      ""  r  o  s
""     0  1  2  3
h      1  1  2  3
o      2  2  1  2
r      3  2  2  2
s      4  3  3  2
e      5  4  4  3   -> 3
```

Java Solution

```java
public class EditDistance {
    public int minDistance(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++)
                dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? dp[i - 1][j - 1]
                        : 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
        return dp[m][n];
    }
}
```

Complexity

- Time: O(m n). Space: O(m n) (O(n) with rolling rows).

Pitfalls

- Base row/column represent all-insert / all-delete costs.